Quiz 2 Solutions

Question	Parts	Points
1: True or False	10	30
2: Mo Money, Mo Flow!	4	20
3: The Dual: There And Back Again	4	20
4: MIN-SAT	4	20
5: We're All In This Together	6	30
Total:		120

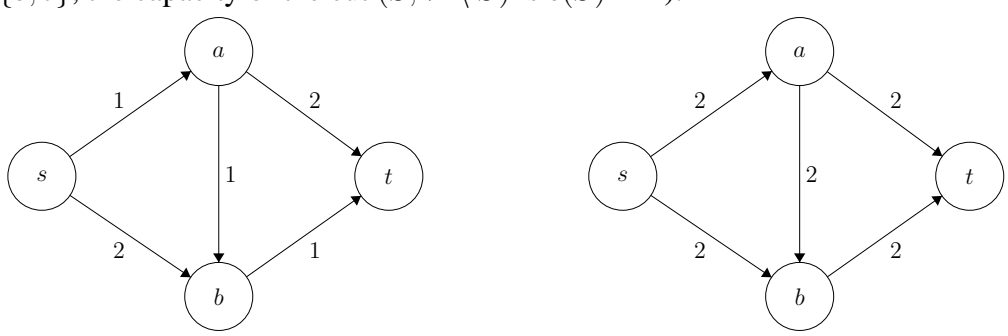
Name: _____

Problem 1. [30 points] **True or False** (10 parts)
Please fill in the bubble next to **True** or **False** for the following.

No justification is needed (nor will be considered).

(a) [3 points] ◯ **True** ◯ **False**
Let $G = (V, E, s, t, c)$ be a finite, simple, directed flow network. Consider an s-t path in G. If the capacity of every edge along the path is increased by $\Delta > 0$, then the value of a maximum flow in G increases by exactly Δ.

Solution: False. Consider the flow network shown below on the left. The value of a maximum flow in this flow network is 2 (we can send a flow of value 1 along $s \to a \to t$ and a flow of value 1 along $s \to b \to t$, additionally, if $S = \{s, b\}$, the capacity of the cut $(S, V \setminus S)$ is $c(S) = 2$). Suppose we increase the capacity of every edge along the s-t path $s \to a \to b \to t$ by $\Delta = 1 > 0$. We then get the flow network on the right. The value of a maximum flow in this flow network is 4, i.e., it increases by $2 \neq 1 = \Delta$ (we can send a flow of value 2 along $s \to a \to t$ and a flow of value 2 along $s \to b \to t$, additionally, if $S = \{s, b\}$, the capacity of the cut $(S, V \setminus S)$ is $c(S) = 4$).



(b) [3 points] ◯ **True** ◯ **False**
Let $G = (V, E, s, t, c)$ be a finite, simple, directed flow network. Consider a minimum s-t cut of G. If the capacity of every edge crossing the cut is halved, then the value of a maximum flow in G exactly halves.

Solution: True. Let f^* be a maximum flow and $(S, V \setminus S)$ be a minimum s-t cut of G. Initially, we have $c(S) = |f^*|$. If the capacity of every edge crossing S is halved, $c(S) = \frac{|f^*|}{2}$. This implies that the value of a minimum cut of G is now at most $\frac{|f^*|}{2}$, and hence the value of a maximum flow in G becomes at most $\frac{|f^*|}{2}$. However, G still admits a flow of value $\frac{|f^*|}{2}$, namely $\frac{f^*}{2}$ where we halve the flows on all edges. Therefore, the value of a maximum flow in G becomes $\frac{|f^*|}{2}$.

(c) [3 points] ◯ **True** ◯ **False**

Let $G = (V, E, s, t, c)$ be a finite, simple, directed flow network. If every edge crosses at least one minimum s-t cut of G, then G has a unique maximum flow.

Solution: True. Consider a maximum flow f^* of G and an edge $e \in E$. Since e crosses a minimum cut, $c_{f^*}(e) = 0$. This is because if $c_{f^*}(e) > 0$, then the flow across the cut would be smaller than the capacity of the cut, contradicting either the fact that f^* is a maximum flow, or that it is a minimum cut. Since every edge crosses some minimum cut, $c_{f^*}(e) = 0$ for all $e \in E$. This means that $f^*(e) = c(e)$ for all $e \in E$. This implies that the maximum flow in G is unique and is given by $f^*(e) = c(e)$ for all $e \in E$.

(d) [3 points] ◯ **True** ◯ **False**

Let $A \in \mathbb{R}^{m \times n}$, $\vec{b}_L, \vec{b}_R \in \mathbb{R}^m$, $\vec{c} \in \mathbb{R}^n$, and $\vec{x} \in \mathbb{R}^n$, where $m, n \geq 1$.

Consider the two linear programs $\mathcal{P}_L$ and $\mathcal{P}_R$:

$$\mathcal{P}_L: \quad \min \ \vec{c}^T \vec{x} \qquad A\vec{x} \geq \vec{b}_L \qquad \vec{x} \geq \vec{0}$$

$$\mathcal{P}_R: \quad \min \ \vec{c}^T \vec{x} \qquad A\vec{x} \geq \vec{b}_R \qquad \vec{x} \geq \vec{0}$$

For some A, $\vec{b}_L, \vec{b}_R$, and $\vec{c}$, the LP $\mathcal{P}_L$ has a finite optimal value while the LP $\mathcal{P}_R$ is unbounded.

Solution: False. Consider the dual programs $\mathcal{D}_L$ and $\mathcal{D}_R$ of $\mathcal{P}_L$ and $\mathcal{P}_R$ respectively.

$$\mathcal{D}_L: \quad \max \ \vec{b}_L^T \vec{y} \qquad A^T \vec{y} \leq \vec{c} \qquad \vec{y} \geq \vec{0}$$

$$\mathcal{D}_R: \quad \max \ \vec{b}_R^T \vec{y} \qquad A^T \vec{y} \leq \vec{c} \qquad \vec{y} \geq \vec{0}$$

Note that $\vec{y} \in \mathbb{R}^m$ in the above. If $\mathcal{P}_L$ has a finite optimal value, by strong duality, so does $\mathcal{D}_L$. This in particular means that $\mathcal{D}_L$ is feasible. But $\mathcal{D}_L$ and $\mathcal{D}_R$ have the same feasible region and hence $\mathcal{D}_R$ must be feasible too. This means that $\mathcal{P}_R$ cannot be unbounded, as otherwise, $\mathcal{D}_R$ would be infeasible.

(e) [3 points] ◯ **True** ◯ **False**

Let $A \in \mathbb{R}^{m \times n}$, $\vec{b}_L, \vec{b}_R \in \mathbb{R}^m$, $\vec{c} \in \mathbb{R}^n$, and $\vec{x} \in \mathbb{R}^n$, where $m, n \geq 1$.

Consider the two linear programs $\mathcal{P}_L$ and $\mathcal{P}_R$ defined in part (d).

For some A, $\vec{b}_L, \vec{b}_R$, and $\vec{c}$, the LP $\mathcal{P}_L$ has a finite optimal value while the LP $\mathcal{P}_R$ is infeasible.

Solution: True. Consider the linear programs $\mathcal{P}_L$ and $\mathcal{P}_R$ as below:

$$\mathcal{P}_L: \quad \begin{aligned} \min\ & x_1 \\ -x_1 &\geq -1 \\ x_1 &\geq 0 \end{aligned} \qquad\qquad \mathcal{P}_R: \quad \begin{aligned} \min\ & x_1 \\ -x_1 &\geq 1 \\ x_1 &\geq 0 \end{aligned}$$

(f) [3 points] ◯ **True** ◯ **False**

Let $A \in \mathbb{R}^{m \times n}$, $\vec{b} \in \mathbb{R}^m$, $\vec{c} \in \mathbb{R}^n$, and $\vec{x} \in \mathbb{R}^n$, where $m, n \geq 1$.

Consider the linear program $\mathcal{P}_=$:

$$\mathcal{P}_=: \quad \begin{aligned} \min\ & \vec{c}^T \vec{x} \\ A\vec{x} &= \vec{b} \\ \vec{x} &\geq \vec{0} \end{aligned}$$

For some A, $\vec{b}$, and $\vec{c}$, the LP $\mathcal{P}_=$ has a finite optimal value while its dual is infeasible.

Solution: False. Strong duality holds regardless of whether or not the LP is written in standard form. Hence, if $\mathcal{P}_=$ has a finite optimal value, so does its dual.

(g) [3 points] ◯ **True** ◯ **False**

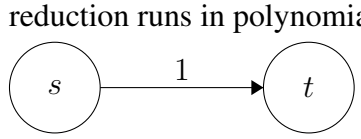
Let INTEGRAL-FLOW be the following decision problem:

> **INTEGRAL-FLOW:** Given a finite, simple, directed flow network $G = (V, E, s, t, c)$ and integer k, does G have an **integral** flow f such that $|f| \geq k$?

Note that a flow f is an **integral** flow if and only if $f(u, v)$ is an integer for all $u, v \in V$. Also, the capacities given by c may **not** be integers.

Assume **P** = **NP**. Then, INTEGRAL-FLOW is **NP**-complete.

Solution: True. If **P** = **NP**, we can show that CNF-SAT $\leq_p$ INTEGRAL-FLOW. Suppose we are given an instance F of CNF-SAT. Since CNF-SAT $\in$ **NP** = **P**, we can solve F in polynomial time. Once we do so, if F is a YES instance of CNF-SAT, we output the instance $\langle G, 1 \rangle$ of INTEGRAL-FLOW, where G is the flow network given below. If F is a NO instance of CNF-SAT, we output the instance $\langle G, 2 \rangle$ of INTEGRAL-FLOW. It is straightforward to see that this reduction runs in polynomial time and is correct.



(h) [3 points] ◯ **True** ◯ **False**

Let INTEGRAL-FLOW be the decision problem defined in part (g).

Assume **P** $\neq$ **NP**. Then, INTEGRAL-FLOW is **NP**-complete.

Solution: False. INTEGRAL-FLOW can be solved in polynomial time and hence belongs to **P**. If it were **NP**-complete, then **P** $=$ **NP**, which contradicts the given assumption. To see that INTEGRAL-FLOW can be solved in polynomial time, note that for any edge $e \in E$, we can simply replace $c(e)$ by $\lfloor c(e) \rfloor$ and compute a maximum flow using Edmonds-Karp. Since the capacities are all integers, from flow integrality, we know that there exists an integral maximum flow. Furthermore, since we are only interested in the space of integral flows, the flow along any edge must be upper bounded by $\lfloor c(e) \rfloor$, and so our modification does in fact produce the value of a maximum integral flow in G. Once we compute a maximum flow, we simply compare its value to k and output YES if and only if its value is at least k. Since our modification of the capacities and Edmonds-Karp both run in polynomial time, the claim follows.

(i) [3 points] ◯ **True** ◯ **False**

Let COUNT-CNF-SAT be the following decision problem:

COUNT-CNF-SAT: Given a CNF formula F and integer t, does F have at least t satisfying assignments?

We can conclude that COUNT-CNF-SAT is in **NP** on the grounds that a valid certificate for a YES instance $\langle F, t \rangle$ of COUNT-CNF-SAT is:

- a set of t satisfying assignments of F if $t > 0$, and
- the empty string (i.e., a blank certificate) if $t \leq 0$.

Solution: False. There are many ways to see this, and here is one of them. Suppose F has n variables and m clauses. A set of t distinct satisfying assignments of F will be of length $\Omega(nt)$, while the size of the instance $\langle F, t \rangle$ is $O(nm + \log t)$. It is therefore possible that say $m = O(n)$, $t = \Theta(2^n)$, and hence the size of the certificate would be $\Omega(n \cdot 2^n)$ while the size of the instance would be $O(n^2)$, i.e., the size of the certificate is not polynomial in the size of the instance. For example, consider $F = x_1 \vee \ldots \vee x_n$ and $t = 2^n - 1$. Th certificate described in the problem would list all Boolean assignments to $x_1, \ldots, x_n$ except for $x_1 = \ldots x_n =$ FALSE.

(j) [3 points] ◯ **True** ◯ **False**

Recall that CNF-SAT is the following decision problem:

> **CNF-SAT:** Given a CNF formula F, does F have a satisfying assignment?

Let COUNT-CNF-SAT be the decision problem defined in part (i).

We can conclude that COUNT-CNF-SAT is **NP**-hard because CNF-SAT $\leq_p$ COUNT-CNF-SAT.

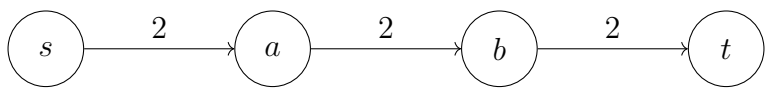
Solution: True. Trivially, CNF-SAT $\leq_p$ COUNT-CNF-SAT. Given an instance F of CNF-SAT, we create the instance $\langle F, 1\rangle$ of COUNT-CNF-SAT. It is straightforward to see that this reduction runs in polynomial time and is correct. Since CNF-SAT is **NP**-complete, it is **NP**-hard, and hence so is COUNT-CNF-SAT.

Problem 2. [20 points] **Mo Money, Mo Flow!** (4 parts)
Let $G = (V, E, s, t, c)$ be a finite, simple, directed flow network.

You want to increase the value of a maximum flow in this network. To do so, you are allowed to perform the following operation: For the price of one coin, you can choose any edge $e \in E$ in this network, and increase its capacity by 1.

You want to find out the **minimum** number of coins you must pay in order to increase the value of a maximum flow in G.

(a) [2 points] As a warm-up, consider the following flow network G (the numbers on top of the edges denote their capacities):



Flow network G.

What is the minimum number of coins you must pay in order to increase the value of a maximum flow in G?

No justification is needed (nor will be considered).

Solution: 3. The capacities of each of the three edges (s, a), (a, b), and (b, t) must be increased in order for the value of a maximum flow in G to increase from 2. This requires a minimum of 3 coins to be paid.

(b) [6 points] Let G be a general flow network as described in the problem. Provide an algorithm that checks if 1 coin is enough to increase the value of a maximum flow in G. Briefly justify the correctness and runtime of your algorithm.

For full credit, your algorithm must run in $O(nm^2)$ time, where $|V| = n$ and $|E| = m$.

Solution: Let f^* be a maximum flow in G. For each edge $e \in E$, consider the effect of increasing the capacity of e by 1. This would mean that in the residual network G_{f^*}, the residual capacity $c_{f^*}(e)$ of e would also increase by 1. By definition, the maximum flow in G can be increased by paying 1 coin if and only if there is an augmenting path in G_{f^*} as a result of increasing the capacity of some $e \in E$ by 1. This is equivalent to there being an edge $e = (u, v) \in E$ such that $u \in S$ and $v \in T$, where

$$S = \{v \subseteq V : v \text{ is reachable from } s \text{ in } G_{f^*}\}$$

and

$$T = \{v \subseteq V : t \text{ is reachable from } v \text{ in } G_{f^*}\}$$

Therefore, we first determine a maximum flow f^* in G using Edmonds-Karp in $O(nm^2)$ time. We then compute S and T as defined above using a BFS in $O(n+m)$ time and check whether there exists an edge $e = (u, v) \in E$ such that $u \in S$ and $v \in T$ in $O(m)$ time. If

so, we output YES, and otherwise, NO. The correctness of the algorithm follows from the previous discussion. The runtime of the algorithm is $O(nm^2 + n + m + m) = O(nm^2)$.

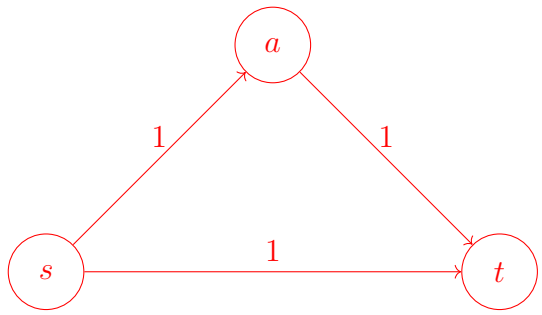
Alternative Solution: After computing f^*, instead of computing S and T as above, one could iterate through each edge, try increasing its (residual) capacity by 1, and then just run a DFS or BFS to check if there is a path from s to t in $O(n+m)$ time. This achieves a total runtime of $O(nm^2 + m(n+m))$, which still falls within the required runtime bound.

Alternative Solution: Instead of checking every edge, checking only the edges crossing a minimum cut is also sufficient. To see this, note that if an edge e has its capacity is increased and the value of the maximum flow increases as a result, then the capacity of the minimum cut must also increase by the Max-Flow Min-Cut Theorem. Therefore, every minimum cut must contain e. As a result, any minimum cut we find must also contain e, so iterating through only the edges in a minimum cut is sufficient. Since finding a minimum cut from f^* would take $O(n + m)$ time and the number of edges in the minimum cut is still $O(m)$, this doesn't save any time asymptotically, but still restricts the search space.

Alternative Solution: Similar to the previous alternative solution, increasing the weight of **all** edges crossing any minimum cut at the same time and checking if there are any new s-t paths is sufficient as well. To see this, we claim that one coin is not sufficient if and only if there exists two edge-disjoint minimum cuts. The proof of the claim follows from a similar reasoning to the previous alternative solution. Therefore, if we increase the capacity of all edges in a minimum cut and the value of the maximum flow stays the same, then there exists two edge-disjoint minimum cuts, and we can conclude that one coin is not sufficient.

Common mistakes:

- Some students attempted to modify Edmond's Karp by checking for each path if the edges with the least and second-least capacity are different and returning true if they are. While this solution works in some cases, there are cases (i.e. if there are no augmenting paths to begin with) where this solution fails, so this only received partial credit.
- Some students attempted the following: compute a minimum cut of the graph and increase the capacity of a random edge along that minimum cut. If this leads to a new s-t path, then output YES. However, this fails when the minimum cut is not unique. Consider a flow network with two disjoint s-t paths given by $s \to t$ and $s \to a \to t$ where all edge capacities are 1 (shown on the next page). The answer is YES as we can increase the capacity of the $s \to t$ edge. However a minimum cut will include exactly one of $s \to a$ or $a \to t$, and increasing the capacity of either of these will not lead to a new s-t path.



- Solutions that just stated "check if there is a path with only one edge with residual capacity 0 using BFS" were only given partial credit if it was not specified how this BFS would run. One possibility would be to use graph duplication.
- Solutions that attempt to iterate through all s-t paths don't fit within the runtime constraint because the number of paths between two vertices in a graph is not necessarily polynomial in the size of the graph.

(c) [8 points] Let G be a general flow network as described in the problem. Let f^* be a maximum flow in G and let $G_{f^*} = (V, E_{f^*}, s, t, c_{f^*})$ be the corresponding residual network. Define the finite, simple, directed, edge-weighted graph $G' = (V, E, w)$, where for $e \in E$,

$$w(e) = \begin{cases} 1 & \text{if } c_{f^*}(e) = 0 \\ 0 & \text{otherwise} \end{cases}$$

Prove that the minimum number of coins you must pay in order to increase the value of a maximum flow in G is equal to the weight of a lightest s-t path in G'.

Solution: Let p be a lightest s-t path in G'. Let

$$A_p = \{e \in p : c_{f^*}(e) = 0\}$$

By definition, the weight of p, $w(p)$, is equal to $|A_p|$. Consider the effect of increasing the capacity of all $e \in A_p$ by 1. This would mean that in the residual network G_{f^*}, the residual capacity $c_{f^*}(e)$ of all $e \in A_p$ would also increase by 1. As a result of this, all edges in p will have a positive residual capacity, i.e., there will be an augmenting path in G_{f^*}, and hence the value of a maximum flow in G would have increased. Therefore, by paying $w(p) = |A_p|$ coins, we can increase the value of a maximum flow in G. This proves that the minimum number of coins one must pay to increase the value of a maximum flow in G is at most $w(p)$.

Now, suppose that we can pay less than $w(p)$ coins and increase the value of a maximum flow in G. This means that there exists a set A' of less than $w(p)$ edges such that the value of a maximum flow in G increases when we increase the capacities of the edges in A' by 1. Consider the effect of increasing the capacity of all $e \in A'$ by 1. This would mean

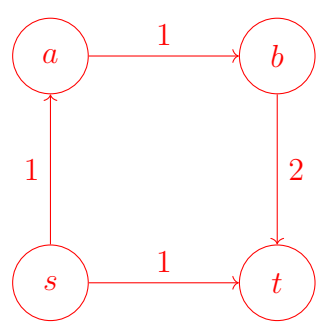
that in the residual network G_{f^*}, the residual capacity $c_{f^*}(e)$ of all $e \in A'$ would also increase by 1. Now, since the value of a maximum flow in G has increased, there will be an augmenting path p' in G_{f^*}. Consider the edges in p'. For any $e \in p' \setminus A'$, $c_{f^*}(e)$ is unchanged and if p' is an augmenting path, it must have been the case that $c_{f^*}(e) > 0$ prior to us increasing the capacities of the edges in A' by 1. Therefore, the number of edges in p' with a residual capacity of 0 prior to us increasing the capacities of the edges in A' by 1 is at most $|p' \cap A'| \leq |A'| < w(p)$. This means that $w(p') < w(p)$, contradicting the fact that p is a lightest path from s to t in G'. This proves that the minimum number of coins one must pay to increase the value of a maximum flow in G is at least $w(p)$.

Combining, this proves that the minimum number of coins one must pay to increase the value of a maximum flow in G is equal to $w(p)$.

Common mistakes:

- Many students said something like "if an s-t path has weight x in G', we can push additional flow through this path using x coins; so we should use the lightest path to use the fewest number of coins". For this argument to work, you first need to argue that creating such an augmenting path is the only way to increase the flow (you should argue this by Max-Flow Min-Cut Theorem); many students didn't.
  Note: This corresponds to most submissions getting the "Only shows one direction (that the weight of lightest s-t path is $\geq$ or $\leq$ the minimum number of coins needed)".

- Some students wrote that we want to maximize the number of unsaturated edges on the s-t path, but this is not true, because this is not the same as wanting to minimize the number of saturated edges (s-t paths may have different lengths), which is what we want to do. Consider the flow network shown below with two disjoint s-t paths given by $s \to t$ and $s \to a \to b \to t$. The s-t path given by $s \to t$ has no unsaturated edges, while the $s \to a \to b \to t$ path has 1. However, if we select the $s \to a \to b \to t$ path, we will have to pay 2 coins to increase the flow along it, while if we pick the $s \to t$ path, we only have to pay 1 coin.



- Many students argued that we need to increase the capacity of an edge in each minimum cut, and that the lightest s-t path does that. This needs a proof, particularly because the number of minimum cuts can be exponential in the size of the graph.

(d) [4 points] Let G be a general flow network as described in the problem. Provide an algorithm that determines the minimum number of coins you must pay in order to increase the value of a maximum flow in G. Briefly justify the runtime of your algorithm.

For full credit, your algorithm must run in $O(nm^2)$ time, where $|V| = n$ and $|E| = m$.

Note: *You need **not** justify the correctness of your algorithm.*

Solution: From part (c), our algorithm computes a maximum flow f^*, defines G', and computes the weight of a lightest path from s to t in G' and outputs it. Since f^* can be found in $O(nm^2)$ time using Edmonds-Karp, G' can be defined in $O(n + m)$ time, and the weight of a lightest path from s to t in G' can be found in $O(m + n \log n)$ time using Dijkstra, our algorithm runs in $O(nm^2)$ time.

Alternative Solution: The lightest s-t path in G' may also be found using 0-1 BFS in $O(m + n)$ time or Bellman-Ford in $O(mn)$ time.

Common mistakes:

- Some students tried to find a maximum flow using Ford-Fulkerson or maximum bottleneck capacity augmenting paths, neither of which guarantee a runtime of $O(nm^2)$.
- Some students did not specify how to find the lightest s-t path in G'.
- Some students tried to find the lightest s-t path in G' using generic BFS. However, BFS for shortest paths only works for unweighted graphs, whereas G' has edges with weights of both 0 and 1. It is necessary to modify BFS to handle this, by pushing vertices reached via 0-weight edges to the front of the queue (this is known as 0-1 BFS). Some students tried to get around this by removing the 0-weight edges. However, the lightest s-t path may contain 0-weight edges, and this would prevent us from finding it!
- Some students tried to use Edmonds-Karp on G' or modify Edmonds-Karp directly to find the lightest s-t path. However, Edmonds-Karp returns the maximum flow of the graph, not the shortest path! It is also important to recognize that shortest augmenting paths in the residual graph of G do not correspond to shortest paths in G'.
- Some students tried to find the lightest s-t path in G' by brute-forcing over all s-t paths. This does not guarantee a runtime of $O(nm^2)$.

Problem 3. [20 points] **The Dual: There And Back Again** (4 parts)
We will derive two interesting facts using and about duality. Let $A \in \mathbb{R}^{m \times n}$, $\vec{b} \in \mathbb{R}^m$, $\vec{c} \in \mathbb{R}^n$, and $\vec{y} \in \mathbb{R}^m$, where $m, n \geq 1$. Consider the linear program $\mathcal{P}$:

$$\begin{aligned} \mathcal{P}: \quad & \max \ \vec{b}^T \vec{y} \\ & A^T \vec{y} \leq \vec{c} \end{aligned}$$

Notice that $\mathcal{P}$ does not have any non-negativity constraints.

(a) [4 points] Using decision variables $\vec{x} \in \mathbb{R}^n$, write the dual LP $\mathcal{D}$ of the LP $\mathcal{P}$.
Note: *Your dual LP need* ***not*** *be in standard form.*

Solution: The dual LP $\mathcal{D}$ is as follows:

$$\begin{aligned} \mathcal{D}: \quad & \min \ \vec{c}^T \vec{x} \\ & A\vec{x} = \vec{b} \\ & \vec{x} \geq \vec{0} \end{aligned}$$

Common mistakes:

- Some students wrote $A\vec{x} \geq \vec{b}$ or $A\vec{x} \leq \vec{b}$, and/or omitted the non-negativity constraints.

(b) [8 points] Prove that there exists $\vec{x} \in \mathbb{R}^n$ such that $A\vec{x} = \vec{b}$ and $\vec{x} \geq \vec{0}$ **if and only if** there does **not** exist $\vec{y} \in \mathbb{R}^m$ such that $A^T\vec{y} \leq \vec{0}$ and $\vec{b}^T\vec{y} > \vec{0}$.[1]
Hint: *Try using duality for $\mathcal{P}$ and $\mathcal{D}$ from part (a), with $\vec{c} = \vec{0}$. Is $\mathcal{P}$ feasible when $\vec{c} = \vec{0}$?*

Solution: Following the hint, we consider the linear program $\mathcal{P}_{\vec{0}}$ and its dual $\mathcal{D}_{\vec{0}}$:

$$\begin{aligned} \mathcal{P}_{\vec{0}}: \quad & \max \ \vec{b}^T \vec{y} \\ & A^T \vec{y} \leq \vec{0} \end{aligned} \qquad\qquad \begin{aligned} \mathcal{D}_{\vec{0}}: \quad & \min \ \vec{0}^T \vec{x} \\ & A\vec{x} = \vec{b} \\ & \vec{x} \geq \vec{0} \end{aligned}$$

Suppose there exists $\vec{x} \in \mathbb{R}^n$ such that $A\vec{x} = \vec{b}$ and $\vec{x} \geq \vec{0}$. This means that $\mathcal{D}_{\vec{0}}$ is feasible. Also, $\mathcal{D}_{\vec{0}}$ is clearly bounded as the value of its objective function is always 0. By strong duality, $\mathcal{P}_{\vec{0}}$ is also feasible and bounded with an optimal objective function value of 0, i.e., for any $\vec{y} \in \mathbb{R}^m$, if $A^T\vec{y} \leq \vec{0}$, then $\vec{b}^T\vec{y} \leq 0$, i.e., there does not exist $\vec{y} \in \mathbb{R}^m$ such that $A^T\vec{y} \leq \vec{0}$ and $\vec{b}^T\vec{y} > \vec{0}$.
Suppose there does not exist $\vec{x} \in \mathbb{R}^n$ such that $A\vec{x} = \vec{b}$ and $\vec{x} \geq \vec{0}$. This means that $\mathcal{D}_{\vec{0}}$ is infeasible. Also, $\mathcal{P}_{\vec{0}}$ is clearly feasible as $\vec{0}$ is a feasible solution to $\mathcal{P}_{\vec{0}}$. By strong duality,

[1] **Fun fact**: This is a variant of what is popularly known as *Farkas' Lemma*.

$\mathcal{P}_{\vec{0}}$ is feasible and unbounded, i.e., there exists some $\vec{y} \in \mathbb{R}^m$ such that $A^T\vec{y} \leq \vec{0}$ and $\vec{b}^T\vec{y} > 0$.

Therefore, there exists $\vec{x} \in \mathbb{R}^n$ such that $A\vec{x} = \vec{b}$ and $\vec{x} \geq \vec{0}$ if and only if there does not exist $\vec{y} \in \mathbb{R}^m$ such that $A^T\vec{y} \leq \vec{0}$ and $\vec{b}^T\vec{y} > \vec{0}$.

Alternative Solution: The if and only if directions of the statement can both be proved using the contrapositive.

- Suppose there exists $\vec{y} \in \mathbb{R}^m$ such that $A^T\vec{y} \leq \vec{0}$ and $\vec{b}^T\vec{y} > \vec{0}$. This implies that $\mathcal{P}_{\vec{0}}$ has a feasible solution $\vec{y}$ whose objective value $\vec{b}^T\vec{y}$ is greater than 0. If there exists a feasible solution $\vec{x}$ to $\mathcal{D}_{\vec{0}}$, then by weak duality, $0 = \vec{0}^T\vec{x} \geq b^T\vec{y} > 0$, which is a contradiction. Thus, $\mathcal{D}_{\vec{0}}$ is infeasible, and so there does not exist $\vec{x} \in \mathbb{R}^n$ such that $A\vec{x} = \vec{b}$ and $\vec{x} \geq \vec{0}$.

- Suppose there does not exist $\vec{y} \in \mathbb{R}^m$ such that $A^T\vec{y} \leq \vec{0}$ and $\vec{b}^T\vec{y} > 0$. We know that $\mathcal{P}_{\vec{0}}$ is feasible since $\vec{0}$ is feasible solution. Our supposition implies that $\mathcal{P}_{\vec{0}}$ does not have a feasible solution whose objective is greater than 0, so the optimal value of $\mathcal{P}_{\vec{0}}$ is upper bounded by 0. Since $\mathcal{P}_{\vec{0}}$ is feasible and bounded, by strong duality, $\mathcal{D}_{\vec{0}}$ must be feasible. Thus, there exists $\vec{x} \in \mathbb{R}^n$ such that $A\vec{x} = \vec{b}$ and $\vec{x} \geq \vec{0}$.

Common mistakes:

- Some students proved only one direction, for example, they proved that given that $\mathcal{D}_{\vec{0}}$ is feasible, there does not exist a $\vec{y} \in \mathbb{R}^m$ such that $A^T\vec{y} \leq \vec{0}$ and $\vec{b}^T\vec{y} > 0$.

(c) [4 points] Let $v \in \mathbb{R}$. Consider the linear program $\mathcal{P}_1$:

$$\begin{aligned} \mathcal{P}_1 : \quad \min \quad & -x_1 \\ A\vec{x} &= \vec{b} \\ \vec{c}^T\vec{x} &\leq v \\ \vec{x} &\geq 0 \end{aligned}$$

Write the dual LP $\mathcal{D}_1$ of the LP $\mathcal{P}_1$.

Note: *Your dual LP need* ***not*** *be in standard form.*

Solution: We will use the decision variables $\vec{y} \in \mathbb{R}^m$ for the constraints $A\vec{x} = \vec{b}$ and the

decision variable $z \in \mathbb{R}$ for the constraint $\vec{c}^T\vec{x} \leq v$. The dual LP $\mathcal{D}_1$ is as follows:

$$\mathcal{D}_1: \quad \max \ \vec{b}^T\vec{y} + vz$$
$$A^T\vec{y} + z \cdot \vec{c} \leq \left.\begin{pmatrix} -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}\right\} \text{one } -1 \text{ and } (n-1) \text{ 0s}$$
$$z \leq 0$$

Alternative Solution: Some students gave the following dual LP, which can be obtained by transforming the objective of the primal to from $\min -x_1$ to $\max x_1$:

$$\mathcal{D}_1: \quad \min \ \vec{b}^T\vec{y} + vz$$
$$A^T\vec{y} + z \cdot \vec{c} \geq \left.\begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}\right\} \text{one } 1 \text{ and } (n-1) \text{ 0s}$$
$$z \geq 0$$

Common mistakes:

- Many students got the correct dual, but with incorrect inequalities, for example they had a $\geq$ instead of a $\leq$ for the maximization dual.

(d) [4 points] Suppose the LP $\mathcal{P}$ stated at the very beginning of the problem is feasible and has a finite optimal value of $v \in \mathbb{R}$. Prove that there exist optimal solutions $\vec{y}^*$ to $\mathcal{P}$ and $\vec{x}^*$ to $\mathcal{D}$ from part (a) such that for all $1 \leq i \leq n$,

$$x_i^* = 0 \textbf{ if and only if} \text{ the } i\text{th constraint of } \mathcal{P} \text{ has slack, i.e., } A_i^T\vec{y}^* < c_i$$

where $A_i \in \mathbb{R}^{m \times 1}$ denotes the ith column of A.[2]

Note: *This part may be* ***challenging****.*

Hint: *Try using duality for $\mathcal{P}_1$ and $\mathcal{D}_1$ from part (c).*

Solution: Since $\mathcal{P}$ is feasible and has a finite optimal value of v, by strong duality, so does $\mathcal{D}$. Let $\vec{x}$ and $\vec{y}$ be optimal solutions to $\mathcal{D}$ and $\mathcal{P}$ respectively. By complementary slackness, we know that

$$x_i(A_i^T\vec{y} - c_i) = 0$$

[2] **Fun fact**: This is known as *strict complementary slackness*.

for all $1 \leq i \leq n$. We will first construct a pair of optimal solutions $\vec{x}^{(i)}$ and $\vec{y}^{(i)}$ for each $1 \leq i \leq n$ such that $x_i^{(i)} = 0$ if and only if $A_i^T \vec{y}^{(i)} < c_i$.

Let us consider $i = 1$. If $x_1 = 0$ and $A_1^T \vec{y} < c_1$, we set $\vec{x}^{(1)} = \vec{x}$ and $\vec{y}^{(1)} = \vec{y}$. We do the same if $x_1 > 0$ and $A_1^T \vec{y} = c_1$. The only remaining case is when $x_1 = A_1^T \vec{y} - c_1 = 0$. Consider the linear programs $\mathcal{P}_1$ and $\mathcal{D}_1$. By virtue of $\vec{x}$, $\mathcal{P}_1$ is feasible. Now, $\mathcal{P}_1$ is either bounded with an optimal value of 0, bounded with an optimal value < 0, or unbounded.

Suppose $\mathcal{P}_1$ is bounded with an optimal value of 0. Then, by strong duality, so is $\mathcal{D}_1$. This means that there exist $\vec{y}'$ and $z' \leq 0$ such that $\vec{b}^T \vec{y}' + vz' = 0$ and $A^T \vec{y}' + z' \cdot \vec{c} \leq \begin{pmatrix} -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$.

- If $z' = 0$, then $\vec{b}^T \vec{y}' = 0$ and $A^T \vec{y}' \leq \begin{pmatrix} -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$. We then set $\vec{x}^{(1)} = \vec{x}$ and $\vec{y}^{(1)} = \vec{y} + \vec{y}'$. Note that $A^T \vec{y}^{(1)} = A^T \vec{y} + A^T \vec{y}' \leq \vec{c} + \begin{pmatrix} -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$, i.e., $\vec{y}^{(1)}$ is a feasible solution of $\mathcal{P}$ and $A_1^T \vec{y}^{(1)} < c_1$. Furthermore, $\vec{y}^{(1)}$ is an optimal solution of $\mathcal{P}$ as $\vec{b}^T \vec{y}^{(1)} = \vec{b}^T \vec{y} + \vec{b}^T \vec{y}' = \vec{b}^T \vec{y}$, and $\vec{y}$ is an optimal solution of $\mathcal{P}$. Since $x_1 = 0$, we are done.
- If $z' < 0$, then $\vec{b}^T \vec{y}' = -vz'$, i.e., $\vec{b}^T \left(-\frac{1}{z'} \cdot \vec{y}'\right) = v$. We then set $\vec{x}^{(1)} = \vec{x}$ and $\vec{y}^{(1)} = -\frac{1}{z'} \cdot \vec{y}'$. Note that $A^T \vec{y}^{(1)} = -\frac{1}{z'} \cdot A^T \vec{y}' \leq \vec{c} - \frac{1}{z'} \begin{pmatrix} -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$, i.e., $\vec{y}^{(1)}$ is a feasible solution of $\mathcal{P}$ and $A_1^T \vec{y}^{(1)} < c_1$. Furthermore, $\vec{y}^{(1)}$ is an optimal solution of $\mathcal{P}$ as $\vec{b}^T \vec{y}^{(1)} = v$, the optimal value of $\mathcal{P}$. Since $x_1 = 0$, we are done.

Suppose $\mathcal{P}_1$ is bounded with an optimal value < 0 or unbounded. Then, there exists $\vec{x}'$ such that $A\vec{x}' = \vec{b}$, $\vec{x}' \geq \vec{0}$, $\vec{c}^T \vec{x}' \leq v$, and $x_1' > 0$. By definition, this means that $\vec{x}'$ is optimal for $\mathcal{D}$ and $x_1' > 0$. We then set $\vec{x}^{(1)} = \vec{x}'$ and $\vec{y}^{(1)} = \vec{y}$. Since $A_1^T \vec{y} = c_1$, we are done.

Thus, we have constructed a pair of optimal solutions $\vec{x}^{(1)}$ and $\vec{y}^{(1)}$ such that $x_1^{(1)} = 0$ if and only if $A_1^T \vec{y}^{(1)} < c_1$. We can repeat the exact same argument to construct the solutions $\vec{x}^{(i)}$ and $\vec{y}^{(i)}$ by considering analogous linear programs $\mathcal{P}_i$ and $\mathcal{D}_i$ for all $1 \leq i \leq n$. What remains now is to construct $\vec{x}^*$ and $\vec{y}^*$ as described in the problem. Let

$$I = \left\{ i : x_i^{(i)} > 0 \right\}$$

Define

$$\vec{x}^* = \begin{cases} \frac{1}{|I|} \cdot \sum_{i \in I} \vec{x}^{(i)} & \text{if } |I| > 0 \\ \vec{x} & \text{if } |I| = 0 \end{cases} \quad ; \quad \vec{y}^* = \begin{cases} \frac{1}{n-|I|} \cdot \sum_{i \notin I} \vec{y}^{(i)} & \text{if } |I| < n \\ \vec{y} & \text{if } |I| = n \end{cases}$$

If $|I| = 0$, then $\vec{x}^* = \vec{x}$ is an optimal solution to $\mathcal{D}$.
If $|I| > 0$, since $\vec{x}^{(i)} \geq \vec{0}$ for all $1 \leq i \leq n$, $\vec{x}^* \geq \vec{0}$. Also,

$$A\vec{x}^* = \frac{1}{|I|} \cdot \sum_{i \in I} A\vec{x}^{(i)} = \frac{1}{|I|} \cdot \sum_{i \in I} \vec{b} = \vec{b}$$

and

$$\vec{c}^T \vec{x}^* = \frac{1}{|I|} \cdot \sum_{i \in I} \vec{c}^T \vec{x}^{(i)} = \frac{1}{|I|} \cdot \sum_{i \in I} v = v$$

Therefore, $\vec{x}^*$ is an optimal solution to $\mathcal{D}$. Furthermore, $x_i^* > 0$ if $i \in I$.
Similarly, if $|I| = n$, then $\vec{y}^* = \vec{y}$ is an optimal solution to $\mathcal{P}$.
If $|I| < n$,

$$A^T \vec{y}^* = \frac{1}{n - |I|} \cdot \sum_{i \notin I} A^T \vec{y}^{(i)} \leq \frac{1}{n - |I|} \cdot \sum_{i \notin I} \vec{c} = \vec{c}$$

and

$$\vec{b}^T \vec{y}^* = \frac{1}{n - |I|} \cdot \sum_{i \notin I} \vec{b}^T \vec{y}^{(i)} = \frac{1}{n - |I|} \cdot \sum_{i \notin I} v = v$$

Therefore, $\vec{y}^*$ is an optimal solution to $\mathcal{P}$. Furthermore, $A_i^T \vec{y}^* < c_i$ if $i \notin I$.
In all cases, by complementary slackness, $A_i^T \vec{y}^* = c_i$ if $i \in I$ and $x_i^* = 0$ if $i \notin I$.
Therefore, there exist optimal solutions $\vec{y}^*$ to $\mathcal{P}$ and $\vec{x}^*$ to $\mathcal{D}$ such that for all $1 \leq i \leq n$, $x_i^* = 0$ if and only if $A_i^T \vec{y}^* < c_i$.

Common mistakes:

- This problem was very difficult, and most students only mentioned strong duality or complementary slackness without making significant progress.

Problem 4. [20 points] **MIN-SAT** (4 parts)
Assume that graphs are finite, simple, undirected, and unweighted.

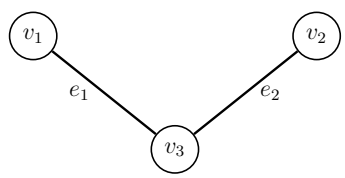
Recall that VERTEX-COVER is the following decision problem:

> **VERTEX-COVER:** Given a graph $G = (V, E)$ and integer K, is there a set of vertices $S \subseteq V$ with $|S| \leq K$ that form a vertex cover, i.e., for any $(u, v) \in E$ either $u \in S$ or $v \in S$ (or both)?

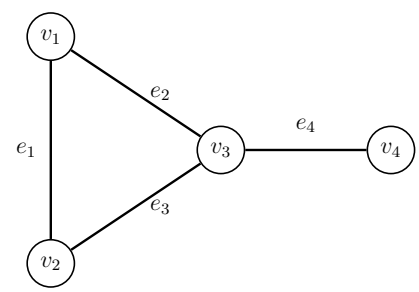
Let MIN-SAT be the following decision problem:

> **MIN-SAT:** Given a CNF formula F and integer t, is there a Boolean assignment to the variables in F that satisfies at most t **clauses** of F?

(a) [4 points] As a warm-up, consider the following graphs G_1, G_2:



Graph G_1.



Graph G_2.

What are the **minimum** possible values K_1, K_2 such that $\langle G_1, K_1 \rangle$ and $\langle G_2, K_2 \rangle$ are both YES instances of VERTEX-COVER?

No justification is needed (nor will be considered).

> **Solution:** $K_1 = 1, K_2 = 2$. Note that G_1 has a vertex cover of size 1, namely $\{v_3\}$, and G_2 has a vertex cover of size 2, namely $\{v_1, v_3\}$. Furthermore, it is easy to see that G_1 has no vertex cover of size 0 and G_2 has no vertex cover of size 1.

(b) [4 points] As a warm-up, consider the following two CNF formulas F_1, F_2:

$$F_1 = x_1 \wedge x_2 \wedge (\neg x_1 \vee \neg x_2)$$

$$F_2 = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3 \vee x_4) \wedge \neg x_4$$

What are the **minimum** possible values t_1, t_2 such that $\langle F_1, t_1 \rangle$ and $\langle F_2, t_2 \rangle$ are both YES instances of MIN-SAT?

No justification is needed (nor will be considered).

Solution: $t_1 = 1, t_2 = 2$. Consider F_1. Regardless of whether we assign x_1 to be either TRUE or FALSE, one of the clauses x_1 and $(\neg x_1 \vee \neg x_2)$ will be satisfied, and therefore $t_1 \geq 1$ if $\langle F_1, t_1 \rangle$ is a YES instance of MIN-SAT. Also, note that if we set $x_1 = x_2 =$ FALSE, then exactly one of the clauses of F_1 will be satisfied. Therefore, $t_1 = 1$.

Consider F_2. Regardless of whether we assign x_1 to be either TRUE or FALSE, one of the clauses $(x_1 \vee x_2)$ and $(\neg x_1 \vee x_3)$ will be satisfied. Similarly, regardless of whether we assign x_4 to be either TRUE or FALSE, one of the clauses $(\neg x_2 \vee \neg x_3 \vee x_4)$ and $\neg x_4$ will be satisfied. Therefore, $t_2 \geq 2$ if $\langle F_2, t_2 \rangle$ is a YES instance of MIN-SAT. Also, note that if we set $x_1 = x_2 = x_4 =$ TRUE and $x_3 =$ FALSE, then exactly two of the clauses of F_2 will be satisfied. Therefore, $t_2 = 2$.

(c) [4 points] Show that MIN-SAT is in **NP**.

Solution: We prove that MIN-SAT is in **NP** by showing that there exists a verifier that validates proofs for YES instances of MIN-SAT in polynomial time. A verifier can interpret the proof as an assignment of values to the variables and output YES if and only if at most t clauses of the formula are satisfied under the assignment. This can be easily checked in polynomial time (in fact, linear time in the total size of the formula), and the size of the proof is also polynomial. Hence, MIN-SAT is in **NP**.

Note that by definition an instance of MIN-SAT is a YES instance if and only if it has a proof as described above on which the verifier described above would output YES.

Common mistakes:

- Some students specified a correct certificate but did not describe how a verifier can verify the certificate.

(d) [8 points] Show that MIN-SAT is **NP**-complete.

Hint: *Try a reduction from* VERTEX-COVER. *Consider relating G_1, G_2 with F_1, F_2 respectively by associating the edge e_i in the graph with the variable x_i in the formula, and the vertex v_j in the graph with the jth clause in the formula.*

Solution: From part (c) we already know that MIN-SAT is in **NP**. To show that MIN-SAT is **NP**-complete, it remains to show that it is also **NP**-hard. Following the hint, we do so by reducing VERTEX-COVER to it.

STEP 1: First we describe how to construct instances of MIN-SAT from instances of VERTEX-COVER in polynomial time. Given an instance $\langle G, K \rangle$ of VERTEX-COVER, create the formula F as follows. Let $G = (V, E)$. Let the vertices in V be numbered (arbitrarily) from 1 through n, where $n = |V|$. Going forward, we will use a vertex v and its number interchangeably. For every edge $(u, v) \in E$, where $u < v$, we introduce the variable $x_{u,v}$ in F. For every vertex $v \in V$, let $u_1 < \ldots < u_{d(v)}$ be the neighbors of v, where $d(v)$ denotes the degree of v in G. Suppose that when

v is arranged along with its neighbors based on their numbers, we obtain the ordering $u_1 < \ldots < u_{p(v)} < v < u_{p(v)+1} < \ldots < u_{d(v)}$, i.e., suppose that among the neighbors of v, $u_1 < \ldots < u_{p(v)}$ are all less than v, while $u_{p(v)+1} < \ldots < u_{d(v)}$ are all greater than v. Then, corresponding to vertex $v \in V$, we create the clause C_v given by

$$C_v = \left(\neg x_{u_1,v} \vee \ldots \vee \neg x_{u_{p(v)},v} \vee x_{v,u_{p(v)+1}} \vee \ldots \vee x_{v,u_{d(v)}}\right)$$

and add C_v to F. Notice that F is a CNF formula. We will set $t = K$. Therefore, $\langle F, t\rangle$ is an instance of MIN-SAT. Creating $\langle F, t\rangle$ takes time linear in the size of $\langle G, K\rangle$ and the size of $\langle F, t\rangle$ is also linear in the size of $\langle G, K\rangle$. We use $\langle F, t\rangle$ as the corresponding instance of MIN-SAT.

Note: *This reduction essentially produces F_1 from G_1 and F_2 from G_2, where the v_is are numbered as i. For example, consider G_1. It has three vertices, v_1, v_2, v_3, which we will number as $1, 2, 3$ respectively. There are two edges, $e_1 = (v_1, v_3)$ and $e_2 = (v_2, v_3)$. We thus create two variables $x_{1,3}$ and $x_{2,3}$ in F_1. Consider vertex v_1. It has one neighbor, v_3, and since $1 < 3$, we create the clause $C_1 = x_{1,3}$ and add it to F_1. Consider vertex v_2. It has one neighbor, v_3, and since $2 < 3$, we create the clause $C_2 = x_{2,3}$ and add it to F_1. Finally, consider vertex v_3. It has two neighbors, v_1 and v_2, and since $1 < 2 < 3$, we create the clause $C_3 = (\neg x_{1,3} \vee \neg x_{2,3})$ and add it to F_1. Thus,*

$$F_1 = x_{1,3} \wedge x_{2,3} \wedge (\neg x_{1,3} \vee \neg x_{2,3})$$

Renaming $x_{1,3}$ with x_1 and $x_{2,3}$ with x_2 shows the correspondence with F_1 in part (b). One can perform the same exercise with G_2 and F_2 as well.

STEP 2: We show $\langle G, K\rangle$ is a YES instance of VERTEX-COVER if and only if $\langle F, t\rangle$ is a YES instance of MIN-SAT. To this end, we establish two implications:

- *If $\langle G, K\rangle$ is a* YES *instance of* VERTEX-COVER*, then $\langle F, t\rangle$ is a* YES *instance of* MIN-SAT. If $\langle G, K\rangle$ is a YES instance of VERTEX-COVER, then there is set of vertices $S \subseteq V$ with $|S| \leq K$ that form a vertex cover for G. We use this fact to come up with an assignment to the variables in F. Specifically, for $(u, v) \in E$ with $u < v$, we set $x_{u,v}$ to TRUE if and only if $u \in S$. The only thing left to prove is that at most $t = K$ clauses of F evaluate to TRUE under this assignment.

  Note that if v has no neighbors, then C_v is empty, i.e., there is no such clause C_v in F. Going forward, we will be concerned with vertices v with at least one neighbor.

  We will show that for $v \notin S$, C_v will not be satisfied under our assignment. Since $|S| \leq K$, this means that at most $t = K$ clauses of F can evaluate to TRUE under this assignment. Consider any $v \notin S$. Let

  $$C_v = \left(\neg x_{u_1,v} \vee \ldots \vee \neg x_{u_{p(v)},v} \vee x_{v,u_{p(v)+1}} \vee \ldots \vee x_{v,u_{d(v)}}\right)$$

  where $u_1 < \ldots < u_{d(v)}$ are the neighbors of v, $d(v)$ denotes the degree of v, and $u_1 < \ldots < u_{p(v)} < v < u_{p(v)+1} < \ldots < u_{d(v)}$. Since $v \notin S$, our assignment

sets $x_{v,u_{p(v)+1}} = \ldots = x_{v,u_{d(v)}} =$ FALSE and hence those literals do not satisfy C_v. Now, consider the edges $(u_1, v), \ldots, (u_{p(v)}, v)$. Since S is a vertex cover, S must cover them. Since $v \notin S$, it must be the case that $u_1, \ldots, u_{p(v)} \in S$. This means that our assignment sets $x_{u_1,v} = \ldots = x_{u_{p(v)},v} =$ TRUE and hence the literals $\neg x_{u_1,v}, \ldots, \neg x_{u_{p(v)},v}$ do not satisfy C_v either. Therefore, if $v \notin S$, C_v is not satisfied under our assignment.

Therefore, $\langle F, t\rangle$ is a YES instance of MIN-SAT.

- *If $\langle F, t\rangle$ is a* YES *instance of* MIN-SAT*, then $\langle G, K\rangle$ is a* YES *instance of* VERTEX-COVER. If $\langle F, t\rangle$ is a YES instance of MIN-SAT, then there is a Boolean assignment to the variables in F such that at most k clauses of F are satisfied under the assignment. We use this fact to come up with a vertex cover for G. Specifically, for $v \in V$, we add v to S if and only if C_v is satisfied. Since at most k clauses are satisfied and $t = K$, $|S| \leq K$. The only thing left to prove is that S is a vertex cover of G.

  Assume for the sake of contradiction that S isn't a vertex cover and that there is an edge $(u, v) \in E$ with $u < v$ such that $u, v \notin S$. This means that neither C_u nor C_v were satisfied under the assignment. However, note that the literal $x_{u,v}$ appears in C_u and the literal $\neg x_{u,v}$ appears in C_v, and at least one of them must evaluate to TRUE, regardless of the assignment to the variable $x_{u,v}$, giving us our contradiction. Therefore, there does not exist any such edge which is not covered, i.e., S is a vertex cover of G.

  Therefore, $\langle G, K\rangle$ is a YES instance of VERTEX-COVER.

Hence, MIN-SAT is **NP**-hard, and hence **NP**-complete.

Alternative Solution: A similar reduction can be performed as follows.

STEP 1: Given an instance $\langle G, K\rangle$ of VERTEX-COVER, create the formula F as follows. Let $G = (V, E)$. Arbitrarily direct the edges in E. For every edge $e \in E$, we introduce the variable x_e in F. For every vertex $v \in V$ and edge $e \in E$ incident to v, define

$$\ell_v(e) = \begin{cases} x_e & \text{if } e \text{ is an incoming edge of } v \\ \neg x_e & \text{if } e \text{ is an outgoing edge of } v \end{cases}$$

Corresponding to vertex $v \in V$, we create the clause C_v given by

$$C_v = \bigvee_{e \text{ incident to } v} \ell_v(e)$$

and add C_v to F. We will set $t = K$ and use $\langle F, t\rangle$ as our instance of MIN-SAT.

STEP 2: We show $\langle G, K\rangle$ is a YES instance of VERTEX-COVER if and only if $\langle F, t\rangle$ is a YES instance of MIN-SAT.

- If $\langle G, K\rangle$ is a YES instance of VERTEX-COVER, then there is set of vertices $S \subseteq V$ with $|S| \leq K$ that form a vertex cover for G. For $v \in S$, iteratively assign all unassigned literals of C_v to TRUE. Then, arbitrarily assign all unassigned variables. We will show that for $v \notin S$, C_v will not be satisfied under our assignment. Since $|S| \leq K$, this means that at most $t = K$ clauses of F can evaluate to TRUE. Consider any $v \notin S$. If C_v evaluates to TRUE, there must be a literal $\ell_v(e)$ that evaluates to TRUE. Consider that edge $e = (u, v)$. Note that $u \in S$ as S is a vertex cover. Also, exactly one of $\ell_u(e)$ and $\ell_v(e)$ evaluates to TRUE and hence $\ell_u(e)$ evaluates to FALSE. But this is not possible as $u \in S$ and we would have assigned $\ell_u(e)$ to be TRUE. Therefore, $\langle F, t\rangle$ is a YES instance of MIN-SAT.

- If $\langle F, t\rangle$ is a YES instance of MIN-SAT, then there is a Boolean assignment to the variables in F such that at most k clauses of F are satisfied under the assignment. For $v \in V$, we add v to S if and only if C_v is satisfied. Since at most k clauses are satisfied and $t = K$, $|S| \leq K$. Assume for the sake of contradiction that S isn't a vertex cover and that there is an edge $e = (u, v) \in E$ such that $u, v \notin S$. This means that neither C_u nor C_v were satisfied under the assignment. However, note that exactly one of $\ell_u(e)$ and $\ell_v(e)$ evaluates to TRUE, and hence at least one of C_u and C_v must evaluate to TRUE, giving us our contradiction. Therefore, $\langle G, K\rangle$ is a YES instance of VERTEX-COVER.

Alternative Solution: Another reduction can be performed as follows.

STEP 1: Given an instance $\langle G, K\rangle$ of VERTEX-COVER, create the formula F as follows. Let $G = (V, E)$. For every vertex $v \in V$, we introduce the variable x_v in F. Corresponding to vertex $v \in V$, we create the clause C_v given by

$$C_v = \neg x_v \vee \bigvee_{u \text{ incident to } v} x_u$$

and add C_v to F. We will set $t = K$ and use $\langle F, t\rangle$ as our instance of MIN-SAT.

STEP 2: We show $\langle G, K\rangle$ is a YES instance of VERTEX-COVER if and only if $\langle F, t\rangle$ is a YES instance of MIN-SAT.

- If $\langle G, K\rangle$ is a YES instance of VERTEX-COVER, then there exists a set of vertices $S \subseteq V$ with $|S| \leq K$ that form a vertex cover for G. For $v \in S$, we assign x_v to FALSE; otherwise, we assign x_v to TRUE. We will show that for $v \notin S$, C_v will not be satisfied under our assignment. For contradiction, suppose that C_v is TRUE. Then, there must exist x_u such that it is assigned to be TRUE, which implies that both u and v are not in S, yet there is an edge connecting them, giving us our contradiction. Therefore, $\langle F, t\rangle$ is a YES instance of MIN-SAT.

- If $\langle F, t\rangle$ is a YES instance of MIN-SAT, then there is a Boolean assignment to the variables in F such that at most k clauses of F are satisfied under the assignment. For $v \in V$, we add v to S if and only if C_v is satisfied. Since at most k clauses are satisfied and $t = K$, $|S| \leq K$. Assume for the sake of contradiction that S isn't a vertex cover and that there is an edge $e = (u, v) \in E$ such that $u, v \notin S$. This means that neither C_u nor C_v were satisfied under the assignment. However, note that both of x_u and x_v evaluates to TRUE, and hence both of C_u and C_v must also evaluate to TRUE since C_u contains x_v and C_v contains x_u, giving us our contradiction. Therefore, $\langle G, K\rangle$ is a YES instance of VERTEX-COVER.

Common mistakes:

- Some students added variables to clauses without any negations. This creates a trivial YES instance of MIN-SAT, since one can always set all variables to FALSE.
- For the construction suggested by the hint, the number of satisfied clauses is at most $|S|$, but they are not necessarily equal. For instance, there may be some edges where both incident vertices are included in S. We need to take this into account when assigning the corresponding variables in each clause. It may not be possible to satisfy all the clauses corresponding to the vertices in S. As an illustrative example, consider the graph G with two vertices u and v and one edge connecting them. Even if we take $S = \{u, v\}$ as the vertex cover, the corresponding number of satisfied clauses is 1, not 2, and there is no assignment that satisfies 2 clauses.

Problem 5. [30 points] **We're All In This Together** (6 parts)
Assume that graphs are finite, simple, undirected, and unweighted.

Let $G = (V, E)$ be a graph. An *independent set* of G is a set of vertices $S \subseteq V$ such that no pair of vertices in S are connected by an edge.

Let IND-SET be the following decision problem:

> **IND-SET:** Given a graph $G = (V, E)$ and integer s, is there a set of vertices $S \subseteq V$ with $|S| \geq s$ that form an independent set?

The *maximum independent set* problem, MIS, is the following optimization problem:

> **MIS:** Given a graph $G = (V, E)$, find a set of vertices $S \subseteq V$ that form an independent set of the maximum possible size.

(a) [4 points] Show that IND-SET is in **NP**.

Solution: We prove that IND-SET is in **NP** by showing that there exists a verifier that validates proofs for YES instances of IND-SET in polynomial time. A verifier can interpret the proof as a set $S \subseteq V$ of vertices and output YES if and only if S has at least s distinct vertices and the vertices of S form an independent set (i.e., no pair of vertices in S are connected by an edge). This can be easily checked in polynomial time (in fact, linear time in the total size of G and s), and the size of the proof is also polynomial. Hence, IND-SET is in **NP**.

Note that by definition an instance of IND-SET is a YES instance if and only if it has a proof as described above on which the verifier described above would output YES.

Common mistakes:

- Many students did not point out that the verifier should check that the certificate S has s distinct vertices.
- Some students provided a correct certificate, but didn't specify how they should check that the instance was a YES instance, in particular checking any two vertices in the certificate S don't have an edge.

(b) [4 points] Briefly argue that IND-SET is **NP**-hard.

Solution: We show that IND-SET is **NP**-hard by reducing CLIQUE to it.

STEP 1: First we describe how to construct instances of IND-SET from instances of CLIQUE in polynomial time. Recall CLIQUE from Recitation 8, which we showed is **NP**-complete.

> **CLIQUE:** Given an undirected graph $G = (V, E)$ and integer k, is there a set of vertices $C \subseteq V$ with $|C| \geq k$ that form a *clique*, i.e., a complete subgraph?

Given an instance $\langle G, k\rangle$ of CLIQUE, create the complement graph $G' = (V, E')$, where E' is the set of all possible edges that are not in G. We will set $s = k$. Therefore, $\langle G', s\rangle$ is an instance of IND-SET. Creating $\langle G', s\rangle$ takes time polynomial in the size of $\langle G, k\rangle$.

STEP 2: We show $\langle G, k\rangle$ is a YES instance of CLIQUE if and only if $\langle G', s\rangle$ is a YES instance of IND-SET. To this end, we establish two implications:

- *If $\langle G, k\rangle$ is a* YES *instance of* CLIQUE*, then $\langle G', s\rangle$ is a* YES *instance of* IND-SET. If $\langle G, k\rangle$ is a YES instance of CLIQUE, then there is set of vertices $C \subseteq V$ with $|C| \geq k$ that form a clique in G. We use this fact to come up with an independent set S in G'. Specifically, we set $S = C$. Clearly, $|S| \geq s = k$ as $|S| = |C| \geq k$. The only thing left to prove is that S is an independent set. Consider any pair of vertices in S. Since they belong to C, they must be connected by an edge in G. This means that they are not connected by an edge in G'. Therefore, $\langle G', s\rangle$ is a YES instance of IND-SET.

- *If $\langle G', s\rangle$ is a* YES *instance of* IND-SET*, then $\langle G, k\rangle$ is a* YES *instance of* CLIQUE. If $\langle G', s\rangle$ is a YES instance of IND-SET, then there is set of vertices $S \subseteq V$ with $|S| \geq s$ that form an independent set in G'. We use this fact to come up with a clique C in G. Specifically, we set $C = S$. Clearly, $|C| \geq k = s$ as $|C| = |S| \geq s$. The only thing left to prove is that C is a clique. Consider any pair of vertices in C. Since they belong to S, they are not connected by an edge in G'. This means that they are connected by an edge in G. Therefore, $\langle G, k\rangle$ is a YES instance of CLIQUE.

Hence, IND-SET is **NP**-hard.

Alternative Solution: A simple reduction can also be performed from VERTEX-COVER.

STEP 1: Given an instance $\langle G, K\rangle$ of VERTEX-COVER, where $G = (V, E)$, we will set $s = |V| - K$ and use $\langle G, s\rangle$ as our instance of IND-SET. Creating $\langle G, s\rangle$ takes time linear in the size of $\langle G, K\rangle$.

STEP 2: We show $\langle G, K\rangle$ is a YES instance of VERTEX-COVER if and only if $\langle G, s\rangle$ is a YES instance of IND-SET.

- If $\langle G, K\rangle$ is a YES instance of VERTEX-COVER, then there is set of vertices $S \subseteq V$ with $|S| \leq K$ that form a vertex cover for G. For any pair of vertices $u, v \notin S$, $(u, v) \notin E$ and hence $V \setminus S$ is an independent set in G and $|V \setminus S| \geq |V| - K$. Therefore, $\langle G, s\rangle$ is a YES instance of IND-SET.

- If $\langle G, s\rangle$ is a YES instance of IND-SET, then there is set of vertices $S \subseteq V$ with $|S| \geq |V| - K$ that form an independent set in G. For any edge $e = (u, v) \in E$, at most one of u and v is in S, i.e., at least one of u and v is in $V \setminus S$. This means that $V \setminus S$ is a vertex cover of G and $|V \setminus S| \leq K$. Therefore, $\langle G, K\rangle$ is a YES instance of VERTEX-COVER.

Common mistakes:

- Some students attempted to reduce from problems that are not known to be **NP**-hard, e.g., 2-SAT, 2-COLOR, and MIS (which is an optimization problem).
- Some students stated the relation between another problem and IND-SET but did not give a reduction algorithm, or gave one that reduces IND-SET to that problem. A correct reduction algorithm must reduce that problem to IND-SET.
- Some students omitted or mishandled the integer parameters of the problems.
- Many students did not state that the reduction algorithm runs in polynomial time.
- Some students omitted the proof of correctness for the reduction algorithm or stated that it is obvious. For a reduction from CLIQUE, the proof should at least claim that the *same* vertex set that forms a clique in the initial graph forms an independent set in the complement graph, and vice versa.

(c) [4 points] Let $G = (V, E)$ be a graph. Let $\mathcal{I}_G$ be the following *integer* linear program:

$$\begin{aligned} \mathcal{I}_G: \quad & \max \sum_{v \in V} x_v \\ & x_u + x_v \leq 1 \quad \forall (u, v) \in E \\ & x_v \in \{0, 1\} \quad \forall v \in V \end{aligned}$$

Argue that finding an optimal solution to $\mathcal{I}_G$ is *equivalent* to solving MIS on G.

Solution: Since $x_v \in \{0, 1\}$ for all $v \in V$, we can view them as indicator variables corresponding whether to include v in S. If G has an independent set S with $|S| = s$, then $\mathcal{I}_G$ has a feasible solution of value s given by simply setting $x_v = 1$ if $v \in S$, and $x_v = 0$ otherwise. This constitutes a feasible solution of value s as for any edge $(u, v) \in E$, by definition, either $u \in S$ or $v \in S$, but not both, and therefore $x_u + x_v \leq 1$, and $\sum_{v \in V} x_v = |S| = s$. Similarly, if there is a feasible solution to $\mathcal{I}_G$ of value s, then G has an independent set S of size s given by $S = \{v : x_v = 1\}$. For any edge, since $x_u + x_v \leq 1$ and $x_u, x_v \in \{0, 1\}$, at most one of u, v will be included in S. Since feasible solutions of $\mathcal{I}_G$ are in one-to-one correspondence with independent sets in G, an optimal solution of $\mathcal{I}_G$ will correspond to an independent set in G of the maximum possible size. Therefore, finding an optimal solution to $\mathcal{I}_G$ is *equivalent* to solving MIS on G.

Alternative Solution: Consider a bijection between subsets $S \subseteq V$ of vertices and assignments to x_v for all $v \in V$ as above, where $v \in S \iff x_v = 1$. Many students argued that in this case, since $x_v \in \{0, 1\}$ for all $v \in V$, the constraints on $\mathcal{I}_G$ are *equivalent* to the constraints on an independent set S, i.e., for every edge (u, v), not both of u and v being in S is equivalent to only zero or one, but not both, of x_u and x_v being 1, and that therefore there exists a bijection between independent sets of size s and solutions to $\mathcal{I}_G$ with exactly s variables set to 1.

(d) [5 points] Let $G = (V, E)$ be a graph. Let $\mathcal{P}_G$ be the following linear program:

$$\begin{aligned} \mathcal{P}_G: \quad & \max \sum_{v \in V} x_v \\ & x_u + x_v \leq 1 \quad \forall (u, v) \in E \\ & 0 \leq x_v \leq 1 \quad \forall v \in V \end{aligned}$$

Prove that for every $n > 1$, there exists a graph $G = (V, E)$ with $|V| = n$ and $\frac{\text{OPT}(\mathcal{P}_G)}{\text{OPT}(\mathcal{I}_G)} = \frac{n}{2}$.

Solution: Consider the complete graph $G = (V, E)$ on $n = |V|$ vertices, where $n > 1$. From part (c), $\text{OPT}(\mathcal{I}_G)$ is the size of the maximum independent set in G, which is 1 (one cannot pick any two vertices since every pair of vertices in the complete graph shares an edge). However, note that setting $x_v = \frac{1}{2}$ for all $v \in V$ is a feasible solution to $\mathcal{P}_G$. Therefore, $\text{OPT}(\mathcal{P}_G) \geq \frac{n}{2}$. Also note that if we sum all the inequalities in $\mathcal{P}_G$ for the complete graph G on n vertices, we obtain

$$(n-1) \sum_{v \in V} x_v \leq \binom{n}{2} \implies \sum_{v \in V} x_v \leq \frac{n}{2}$$

Therefore, $\text{OPT}(\mathcal{P}_G) \leq \frac{n}{2}$. Combining, we have $\text{OPT}(\mathcal{P}_G) = \frac{n}{2}$ and hence $\frac{\text{OPT}(\mathcal{P}_G)}{\text{OPT}(\mathcal{I}_G)} = \frac{n}{2}$.

Alternative Solution: Still working with the complete graph G, the upper bound for $\text{OPT}(\mathcal{P}_G)$ can obtained alternatively as follows:

- We have

$$\sum_{v \in V} x_v = \frac{1}{2}\left((x_1 + x_2) + (x_2 + x_3) + \ldots + (x_n + x_1)\right) \leq \frac{n}{2}$$

- Yet another way is as follows. Suppose we have a solution $(x_1, \ldots, x_n)$. Suppose $x_i = \frac{1}{2} + \delta > \frac{1}{2}$ for some i and some $\delta > 0$. Since $x_i + x_j \leq 1$ for all $j \neq i$, $x_j \leq \frac{1}{2} - \delta$. The value of the objective function is at most $\frac{1}{2} + \delta + (n-1)(\frac{1}{2} - \delta) = \frac{n}{2} - (n-2)\delta \leq \frac{n}{2}$ as $n > 1$.

Common mistakes:

- Many students did not justify the upper bound for $\text{OPT}(\mathcal{P}_G)$.

(e) [9 points] Suppose $G = (V, E)$ is a bipartite graph, where V can be partitioned into two sets as $V = A \cup B$ such that vertices in the same set do not have an edge between them.

Construct the finite, simple, directed flow network $G' = (V', E', s, t, c)$ as follows:

- Let $V' = V \cup \{s, t\}$.
- For each edge $(u, v) \in E$, where $u \in A$ and $v \in B$, we add a directed edge from u to v in E' with a capacity of ∞.
- For each $u \in A$, we add a directed edge from s to u in E' with a capacity of 1.
- For each $v \in B$, we add a directed edge from v to t in E' with a capacity of 1.

Let $(X, V' \setminus X)$ be a minimum s-t cut of G'. Prove that $(A \cap X) \cup (B \cap (V' \setminus X))$ is a solution to the MIS problem on G.

Solution: Let S be an independent set in G. We can partition S as $S = S_A \cup S_B$, where $S_A = S \cap A$ and $S_B = S \cap B$. Let $|S_A| = a$ and $|S_B| = b$. Since S is an independent set, there are no edges between S_A and S_B. Therefore, if we consider the cut $(X, V' \setminus X)$, where $X = \{s\} \cup S_A \cup (B \setminus S_B)$, it has a value of $|A| - a + |B| - b = |V| - (a + b)$ (from the edges from s to $A \setminus S_A$ and from $B \setminus S_B$ to t). Therefore, if G has an independent set of size $|S| = |S_A| + |S_B| = a + b$, G' has a cut of value $|V| - (a + b)$.

Similarly, consider any cut $(X, V' \setminus X)$ of G' of finite value. Such a cut must exist as the value of a maximum flow in G' is finite, and hence so is the value of a minimum cut. Let $A \cap X = S_A$ and $B \cap (V' \setminus X) = S_B$. By definition, S_A and S_B are both independent sets in G. Since $c(X)$ is finite, the cut does not contain any of the edges from A to B as they have capacity ∞. Therefore, there are no edges between S_A and S_B. This implies that $S = S_A \cup S_B$ is an independent set in G. Let $|S_A| = a$ and $|S_B| = b$. We have shown that if G' has a cut of value $|A \cap (V' \setminus X)| + |B \cap X| = |A| - a + |B| - b = |V| - (a + b)$, then G has an independent set of size $|S| = |S_A| + |S_B| = a + b$.

Thus, the task of finding a maximum independent set in G is equivalent to finding a minimum cut of G', i.e., if $(X, V' \setminus X)$ is a minimum s-t cut of G', then $(A \cap X) \cup (B \cap (V \setminus X))$ is a solution to the MIS problem on G.

Common mistakes:

- **Note:** To receive credit for the maximality argument, a solution must argue that any independent set can be converted into a cut of corresponding complementary value. Otherwise, it is not clear that the minimum cut produces a maximum independent set as there may be independent sets that cannot be derived from a minimum cut.

- Many students argued that any minimum cut must be of the form $(X, V' \setminus X)$, where $X = \{s\}$ or $X = V' \setminus \{t\}$. This is not necessarily true. Consider the graph G with two specially designated vertices $v_A \in A, v_B \in B$ such that the only edges in G are of the form $(v_A, u), u \in B \setminus \{v_B\}$ and $(u, v_B), u \in A \setminus \{v_A\}$. Then, $c(\{s\}) = |A|$ and $c(V' \setminus \{t\}) = |B|$ but $c(\{s\} \cup (A \setminus \{v_a\}) \cup \{v_B\}) = 2$, and so $\{s\}$ and $V' \setminus \{t\}$ are not minimum cuts.

- Similar to the previous common mistake, it is not necessarily the case that $X \subseteq \{s\} \cup A$ or $X \supseteq \{s\} \cup A$ (the same counterexample applies). Here, we also note that A, B, and X are *fixed*, so we cannot "move" vertices from A to B when arguing that $(A \cap X) \cup (B \cap (V' \setminus X))$ is a maximum independent set.
- Many students argued that the capacity of the cut and value of the maximum flow was equal to the size of the maximum independent set. This is actually false as they are related complementarily, i.e., if the value of the maximum flow is v, then the size of the maximum independent set is $|V| - v$.
- Many students argued that the independent set corresponding to a minimum cut was inclusion-maximal; while this is true, there is no reason why the independent set produced by our maximum flow algorithm has to be entirely contained in any larger independent set.

(f) [4 points] Suppose $G = (V, E)$ is a *tripartite* graph, where V can be partitioned into three sets as $V = A \cup B \cup C$ such that vertices in the same set do not have an edge between them. Assuming that **P** $\neq$ **NP**, is there a polynomial time algorithm that solves MIS on G? Briefly justify your answer.

Solution: We will show that IND-SET on just tripartite graphs is still **NP**-hard. Specifically, let TRI-IND-SET be the following decision problem.

> **TRI-IND-SET:** Given a tripartite graph $G' = (V', E')$ and integer s', is there a set of vertices $S' \subseteq V'$ with $|S'| \geq s'$ that form an independent set?

We will show that IND-SET $\leq_p$ TRI-IND-SET. This shows that TRI-IND-SET is **NP**-hard. This means that if there is a polynomial time algorithm that solves MIS on tripartite graphs, then there is a polynomial time algorithm that solves TRI-IND-SET, which in turn would mean that there is a polynomial time algorithm to solve every problem in **NP**. Therefore, if **P** $\neq$ **NP**, there can be no polynomial time algorithm that solves MIS on tripartite graphs.

We now show that IND-SET $\leq_p$ TRI-IND-SET.

STEP 1: First we describe how to construct instances of TRI-IND-SET from instances of IND-SET in polynomial time.

Given an instance $\langle G, s\rangle$ of IND-SET, create the graph $G' = (V', E')$ as follows. Let $G = (V, E)$. Add all of V into V'. For each edge $e = (u, v) \in E$, introduce two vertices $v_{e,1}, v_{e,2}$ to V' and add the edges $(u, v_{e,1}), (v_{e,1}, v_{e,2}), (v_{e,2}, v)$ to E'. Notice that G' is a tripartite graph. This is because V' can be partitioned as $V' = A \cup B \cup C$ such that every edge crosses the partition as follows. We will let A be all the vertices from V that we added into V', B be all the vertices of the form $v_{e,1}$ that we added, and C be all the vertices of the form $v_{e,2}$. By construction, it is easy to see that all edges cross this partition. We will set $s' = s+m$, where $m = |E|$. Therefore, $\langle G', s'\rangle$ is an instance of TRI-IND-SET. Creating $\langle G', s'\rangle$ takes time linear in the size of $\langle G, s\rangle$ and the size of $\langle G', s'\rangle$ is also linear

in the size of $\langle G, s\rangle$. We use $\langle G', s'\rangle$ as the corresponding instance of TRI-IND-SET.

STEP 2: We show $\langle G, s\rangle$ is a YES instance of IND-SET if and only if $\langle G', s'\rangle$ is a YES instance of TRI-IND-SET. To this end, we establish two implications:

- *If $\langle G, s\rangle$ is a* YES *instance of* IND-SET*, then $\langle G', s'\rangle$ is a* YES *instance of* TRI-IND-SET. If $\langle G, s\rangle$ is a YES instance of IND-SET, then there is set of vertices $S \subseteq V$ with $|S| \geq s$ that form an independent set in G. We use this fact to come up with an independent set S' in G'. Specifically, since $S \subseteq V \subseteq V'$, we begin by adding all of S into S'. Then, for each edge $e = (u, v) \in E$, if $u \in S$, we add $x_{e,2}$ to S', and otherwise add $x_{e,1}$ to S'. Clearly $|S'| \geq s' = s + m$ as $|S'| = |S| \geq s$ and we added m more vertices into S'. Also, S' is an independent set in G'. This is because all edges in G' arise corresponding to edges in G. For any edge $e = (u, v) \in E$, if $u \in S$, then $v \notin S$. Therefore, neither of the neighbors of $x_{e,2}$, namely $x_{e,1}$ and v, are in S'. Similarly, none of the neighbors of u are in S'. Therefore, $\langle G', s'\rangle$ is a YES instance of TRI-IND-SET.

- *If $\langle G', s'\rangle$ is a* YES *instance of* TRI-IND-SET*, then $\langle G, s\rangle$ is a* YES *instance of* IND-SET. If $\langle G', s'\rangle$ is a YES instance of TRI-IND-SET, then there is a set of vertices $S' \subseteq V'$ with $|S'| \geq s'$ that form an independent set in G'. We use this fact to come up with an independent set S in G. First, we slightly modify the set S'. If for any edge $e = (u, v) \in E$, both u and v are in S', we swap v with $x_{e,2}$. Notice that prior to this, neither of $x_{e,1}$ and $x_{e,2}$ could have been in S'. We repeatedly perform such swaps until for every edge $e = (u, v) \in E$, at most one of u and v is in S'. Note that these swaps preserve the fact that S' is an independent set and also preserve its size. Now, we let $S = S' \cap V$. For each edge $e = (u, v) \in E$, at most one of $x_{e,1}$ and $x_{e,2}$ can be in S'. Therefore, by construction, $|S| \geq |S'| - m$. Since $|S'| \geq s + m$, we have $|S| \geq s$. Also, S is an independent set in G as S' had the property that for every edge $e = (u, v) \in E$, at most one of u and v is in S', and therefore this property holds for $S \subseteq S'$ as well. Therefore, $\langle G, s\rangle$ is a YES instance of IND-SET.

Common mistakes:

- Many students tried to argue that the answer is Yes by giving a modified version of the flow algorithm for bipartite graphs from part (e). The reasons why these algorithms don't work vary based on the approach. Note that unless **P** = **NP** there are no polynomial-time algorithms for TRI-IND-SET.

- Some students argued that the flow approach from part (e) does not work for tripartite graphs. While this argument is correct, it only rules out one potential type of polynomial time algorithms for TRI-IND-SET, but not the existence of a polynomial time algorithm in general.

- Some students proposed to reduce IND-SET to TRI-IND-SET by adding two isolated vertices u, v to the graph, claiming it would become a tripartite graph with vertex sets $\{u\}, \{v\}, V$. However, there are edges between vertices in V, so this approach doesn't work.

- Some students assumed that we are just being told that G is tripartite, but not how to partition its vertices into the sets A, B, C. This assumption is reasonable as it wasn't specified that A, B, C would be a part of the problem instance. Given this assumption, they (correctly) argued that it is impossible to find this partition in polynomial time if $\mathbf{P} \neq \mathbf{NP}$ since it is equivalent to finding a solution to 3-COLOR. Thus, they argued, we cannot take advantage of knowing that G is tripartite, so it is as hard as IND-SET. This final conclusion, however, is incorrect. Even without knowing *what* the tripartite partition is, an algorithm may exploit knowing that it *exists* to potentially make the problem easier than IND-SET.